

Synthetic Dataset Construction Pipeline

Contents

1	Pipeline Core Design	1
1.1	Literature overview	2
1.1.1	Generic papers	2
1.1.2	Clinical papers and MIMIC-IV role	2
2	Pipeline	3
2.1	Stage 1: Build a Facet Catalogue	3
2.2	Stage 2: Hidden Query Plans	3
2.3	Stage 3: Generate Structured Fact Rows	4
2.4	Stage 4: Render Fact Rows into Text Chunks via LLM	4
2.5	Stage 5: Create Role-Aware Gold Clusters	5
2.6	Stage 6: Generate Hard Distractor Chunks	5
2.7	Stage 7: Generate the Natural Query and Gold Answer via LLMs	5
2.8	Stage 8: Create Qrels Directly from the Hidden Mapping	6
2.9	Stage 9: Validate the Generated Objects	6
2.10	Stage 10: Embed and Filter by Retrieval Geometry	7
2.11	Stage 11: Evaluate Retrieval Methods	7
3	Data splits	7
4	Prompt Families	8
5	Metrics	9
6	Benchmark Objects	9

1 Pipeline Core Design

The new pipeline reverses the old MIMIC-IV workflow. The new benchmark starts from a structured plan:

1. define query facets and answer structure
2. generate atomic clinical facts that support each facet
3. render those facts into narrative clinical text chunks
4. generate redundant gold clusters and hard distractor clusters
5. generate natural multi-aspect queries from the same hidden plan
6. derive qrels and gold answers directly from the fact–chunk–facet mapping
7. validate the embedding geometry before final evaluation.

1.1 Literature overview

1.1.1 Generic papers

The papers start from an already existing data source and build a QA benchmark on top of it. The main ideas in the papers are: breaking down the data source into structured data supporting knowledge discovery and query generation, allowing for control of query difficulty, explicit validation of the produced synthetic data, explicit evidence provenance, different methods to ensure and validate the diversity of the generated data.

This pipeline borrows their construction methods, but changes the starting point: MIMIC-IV is used only as a reference for terminology, condition families, comorbidity co-occurrences, demographic bins, and note style, but the real starting point is the structured metadata about the dataset we want to generate. The final corpus is generated from a hidden schema so that the retrieval problem has the multi-cluster structure that the raw discharge note corpus did not provide, so it solves by construction many of the challenges encountered in the papers.

Useful ideas from the literature:

- **Evidence-first construction.** MHTS builds questions from claims, clusters, evidence mappings, and difficulty signals [9]. DRAGON stores explicit links among documents, queries, clues, answers, and citations [13]. In this pipeline, those ideas become `query_plans.parquet`, `clinical_facts.parquet`, `chunks.parquet`, and `qrels.parquet`. The LLM writes the clinical text, but the gold labels come from the hidden mapping.
- **Multi-aspect query structure.** ClaimSpect and MQ4CS are useful because they treat a broad user query as a set of separable aspects [6, 1]. Here that decomposition is used before generation: a query plan explicitly chooses facets such as subgroup, treatment duration, rehabilitation outcome, and discharge disposition.
- **Difficulty control.** MHTS motivates storing difficulty as number of facets, number of evidence pieces, and query-evidence similarity [9]. In this benchmark, difficulty is controlled by `n_facets`, `gold_chunks_per_facet`, dominant-cluster size, distractor count, and whether complementary facets are less similar to the surface query.
- **Validation and provenance.** Rahmani et al. and RAGVAL support automatic test-collection construction but also show why validation matters [12, 7]. This pipeline therefore stores fact IDs, facet IDs, cluster IDs, etc.
- **Diversity.** DATAGEN uses attribute-guided generation, diversity-oriented example selection, and group checking to avoid duplicate synthetic examples [5]. Driouich et al. use a diversity agent that clusters and samples source documents for broader RAG coverage [3]. Those methods cannot be copied literally, because this benchmark needs both diversity and planned redundancy. The goal here is to preserve redundancy inside a dominant gold cluster, enforce separation among different gold facets, and create near-miss distractor clusters that are close enough to challenge MMR.
- **Downstream filtering.** Kim et al. argue that synthetic generators should be judged by the downstream behavior their data induces [8]. For the thesis, a generated item is only useful if the candidate pool actually exposes a coverage-sensitive retrieval problem.
- **Optional prompt optimization.** If prompts are unstable, prompt selection can be treated as a multi-objective optimization problem. Pastrían et al. provide one example of this kind of automatic prompt evolution, using evolutionary multi-objective prompt learning rather than manually choosing one prompt by inspection [11]. This is an optional optimization for improving the generator (maybe overkill in this phase).

1.1.2 Clinical papers and MIMIC-IV role

MIMIC-IV can be used as a reference for terminology, frequent conditions, comorbidity distributions, age groups, etc.

This can be strengthened with ideas from the medical papers for synthetic data construction:

- simple care-path templates that say, for a given condition and subgroup, which treatments and outcomes are plausible. This follows the same general idea as HealthBranches [2];
- a small controlled vocabulary of medical terms, so the generator does not randomly alternate between inconsistent names for the same condition, treatment, or outcome. This is the practical part of knowledge-infused medical QA systems such as KI-MAG [15];
- explicit patient-group rules, such as “encephalitis + age over 75 + rehabilitation outcome” or “pneumonia + diabetes + treatment duration”. This borrows the idea of decomposing clinical questions into condition, subgroup, treatment, and outcome constraints [16].

2 Pipeline

2.1 Stage 1: Build a Facet Catalogue

We can build it manually, or using MIMIC-IV as a reference for common conditions, age groups, comorbidities, ... Ideas from the clinical papers can be added later if the generated data is not clinically coherent enough: care-path templates, controlled medical vocabulary, cohort-style criteria, or rubric-style facet checks [2, 15, 16, 4]. These are refinements to be maybe integrated later.

We should have:

- 4–8 primary conditions to use in the benchmark;
- demographic groups, e.g. age < 40, age 40–64, age 65–75, age > 75;
- comorbidity groups, e.g. uncomplicated diabetes, renal disease, chronic pulmonary disease;
- clinical axes we want questions to ask about, e.g. treatment duration, intervention choice, complications, rehabilitation outcome, discharge disposition;
- allowed combinations, so the generator does not create completely implausible cases.

Example catalogue row:

```
condition: encephalitis_or_myelitis
allowed_subgroups:
  demographics: [age_over_75, age_40_64]
  comorbidities: [uncomplicated_diabetes, renal_disease]
allowed_axes:
  - treatment_duration
  - rehabilitation_outcome
  - discharge_disposition
allowed_treatments: [acyclovir, steroids, supportive_care]
allowed_outcomes: [home_rehab, inpatient_rehab, persistent_deficit]
```

2.2 Stage 2: Hidden Query Plans

Generate query plans from the catalogue before writing any query text. A query plan is the hidden recipe saying which facets must be covered. For example, choose:

- one primary condition: encephalitis/myelitis;
- two subgroups to compare: uncomplicated diabetes vs. age > 75;
- two clinical axes: treatment duration and rehabilitation outcome.

This produces four required facets:

```
query_id: Q_001
primary_condition: encephalitis_or_myelitis
query_type: subgroup_comparison
required_facets:
```

```

F1: uncomplicated_diabetes + treatment_duration
F2: uncomplicated_diabetes + rehabilitation_outcome
F3: age_over_75 + treatment_duration
F4: age_over_75 + rehabilitation_outcome
answer_outline:
- summarize treatment duration for uncomplicated diabetes
- summarize rehabilitation outcome for uncomplicated diabetes
- summarize treatment duration for age_over_75
- summarize rehabilitation outcome for age_over_75
- compare the two subgroups
difficulty:
  n_facets: 4
  dominant_gold_cluster: C1
  dominant_gold_chunks: 18
  complementary_gold_chunks_per_cluster: 8
  hard_distractor_chunks: 24
  expected_topk_failure: oversample_dominant_gold_cluster
split: validation

```

The output of this stage is `query_plans.parquet`. This follows the answer-first idea used in synthetic RAG benchmark construction [9, 13]. The important point is that diversity is already specified here: the plan says which clusters are gold, which one is intentionally redundant, which facets are complementary, and which distractor families should be generated.

2.3 Stage 3: Generate Structured Fact Rows

For each required facet, generate several structured fact rows. A fact row is the hidden source of truth for one synthetic case. It says what the generated text is allowed to mention and which facet it supports.

Example:

```

fact_id: FACT_001
query_id: Q_001
facet_ids: [F3, F4]
condition: encephalitis
age_group: age_over_75
comorbidities: [hypertension]
treatment: acyclovir
treatment_duration: 14_days
rehab_outcome: inpatient_rehab_with_persistent_gait_instability
must_mention:
- age over 75
- 14-day antiviral course
- inpatient rehabilitation
- persistent gait instability
must_not_mention:
- uncomplicated diabetes
- complete recovery

```

For each facet, generate enough rows to create a cluster. These rows should vary across synthetic patients while preserving the same facet meaning. The output of this stage is `clinical_facts.parquet`.

2.4 Stage 4: Render Fact Rows into Text Chunks via LLM

Example rendered chunk:

```

The patient, older than 75, was treated for encephalitis with a 14-day
course of acyclovir. At discharge, orientation had improved, but gait

```

```
instability persisted. The patient was transferred to inpatient
rehabilitation for mobility and balance training.
```

Store the generated text together with `fact_ids`, `facet_ids`, `query_id`, `cluster_id`, and `split`. The output of this stage is `chunks.parquet`.

2.5 Stage 5: Create Role-Aware Gold Clusters

Example for Q_001:

- C1: encephalitis + uncomplicated diabetes + treatment duration, 18 chunks, `cluster_role=dominant_gold`;
- C2: encephalitis + uncomplicated diabetes + rehabilitation outcome, 8 chunks, `cluster_role=complementary_gold`;
- C3: encephalitis + age > 75 + treatment duration, 8 chunks, `cluster_role=complementary_gold`;
- C4: encephalitis + age > 75 + rehabilitation outcome, 8 chunks, `cluster_role=complementary_gold`.

The dominant cluster is larger or more lexically similar to the surface query; this creates the top- k failure mode. High similarity inside C1 is allowed, high similarity between C1 and C2 is suspicious, and a chunk that supports several required facets at once should usually be rejected.

2.6 Stage 6: Generate Hard Distractor Chunks

For each query plan, generate non-gold chunks that are plausible and semantically close, but do not answer the required facets. These are needed to distinguish coverage from generic diversity.

Useful distractor types:

- same condition, wrong subgroup: encephalitis without diabetes and not age > 75;
- same subgroup, wrong condition: elderly stroke patients with rehabilitation outcomes;
- same treatment vocabulary, wrong disease: acyclovir duration for shingles;
- same outcome vocabulary, wrong clinical axis: rehabilitation outcome after pneumonia;
- dispersion trap: semantically different clinical text that is novel but does not support any required facet;
- broad background: general encephalitis treatment description.

Store these chunks in `chunks.parquet` with `distractor_type` set and no gold qrel for the target query. This is the practical adaptation of diversity-agent ideas from RAG dataset generation [3]. Each distractor has a reason why it is close to the query and a reason why it is not gold. This poses a challenge for MMR.

2.7 Stage 7: Generate the Natural Query and Gold Answer via LLMs

Example:

```
For patients diagnosed with encephalitis or myelitis, how do treatment durations
and rehabilitative outcomes differ between patients with uncomplicated diabetes
and patients older than 75?
```

Hidden subqueries can be stored for diagnostics, but the main retrieval experiment should use the single natural query [1, 14, 10].

The answer is also produced at this stage, during benchmark construction, before embedding and before any retrieval method is evaluated. The source of truth is the hidden scaffold: `query_plans.parquet`, `clinical_facts.parquet`, and the fact-to-chunk mapping stored in `chunks.parquet`.

Use two separate prompts or procedures:

1. **Query prompt.** Given only the hidden plan, write the user-facing `query_text`. Do not expose the answer wording.
2. **Answer prompt.** Given the required facets, the answer outline, and the structured fact rows grouped by facet, write a canonical answer. The prompt may receive `fact_ids` and their derived `chunk_ids` for citation, but it should not receive distractor chunks or retrieval rankings.

Example answer object:

```
query_id: Q_001
answer_text: >
  In the uncomplicated-diabetes subgroup, treatment was generally shorter,
  with acyclovir courses around 7--10 days, and rehabilitation needs were
  usually limited to home therapy or near-baseline mobility. In patients
  older than 75, treatment was more often extended to about 14 days, and
  discharge commonly involved inpatient rehabilitation because gait
  instability or persistent weakness remained. Overall, the elderly subgroup
  shows longer treatment and worse functional recovery than the diabetes
  subgroup.
facet_answer_summaries:
  F1: diabetes subgroup treatment duration pattern
  F2: diabetes subgroup rehabilitation outcome pattern
  F3: age_over_75 subgroup treatment duration pattern
  F4: age_over_75 subgroup rehabilitation outcome pattern
answer_claims:
- claim_id: A1
  facet_id: F1
  support_fact_ids: [FACT_010, FACT_011]
  support_chunk_ids: [CH_020, CH_021]
- claim_id: A2
  facet_id: F2
  support_fact_ids: [FACT_018, FACT_019]
  support_chunk_ids: [CH_031, CH_032]
```

This is stored in `gold_answers.parquet`.

2.8 Stage 8: Create Qrels Directly from the Hidden Mapping

Create `qrels.parquet` from the known mapping between queries, chunks, facts, and facets.

Example qrels:

query_id	chunk_id	facet_id	relevance_grade	fact_ids
Q_001	CH_017	F3	1	[FACT_001]
Q_001	CH_017	F4	1	[FACT_001]
Q_001	CH_088	none	0	[]

2.9 Stage 9: Validate the Generated Objects

Run simple automatic checks before embedding:

- every query has 3–5 required facets;
- every required facet has enough gold chunks;
- every `cluster_id` has exactly one `cluster_role`;
- every generated chunk can be traced back to fact IDs;
- chunks do not mention forbidden facts;

- distractors do not accidentally answer a gold facet;
- no chunk answers the entire query unless it is part of an easy ablation;
- the canonical answer uses every required facet at least once;

2.10 Stage 10: Embed and Filter by Retrieval Geometry

Embed all chunks and queries. These are benchmark-construction checks, so they are allowed to use the hidden structure of the generated data: `query_id`, `facet_ids`, `cluster_id`, `cluster_role`, `distractor_type`, and `qrels.parquet`. These fields are used to validate the dataset, but they are not exposed to the retrieval methods.

Separate two different questions:

- **Query-local structure, primary generation check.** Use only the chunks generated for the current query. This checks whether the synthetic item itself has the intended facet and cluster structure.
- **Full-corpus visibility, primary retrieval check if full-corpus evaluation is reported.** Use the full `chunks.parquet` corpus. This does not prove the local structure; it only checks whether the gold facets remain visible in the first-stage top- N pool when chunks from other query plans also compete.

Query-local checks:

- each required facet has enough gold chunks;
- intra-facet similarity is high enough that each facet forms a usable cluster;
- inter-facet similarity is lower than intra-facet similarity, so facets do not collapse into one cluster;
- the dominant gold cluster is larger or more query-similar than the complementary gold clusters;
- hard distractors are close to the query or to a gold facet, but have no gold qrel for the target query;

Full-corpus visibility checks, if the main evaluation uses full-corpus retrieval:

- each required facet has at least one gold chunk in the full-corpus top- N pool;
- the top- N pool contains hard distractors or background negatives, so evaluation is not trivial;
- chunks from other query plans do not completely swamp the query’s gold evidence.

Queries that fail these checks can be regenerated or excluded.

2.11 Stage 11: Evaluate Retrieval Methods

The retrieval corpus is part of the benchmark definition, so different corpus choices should be reported as different settings.

- **Full-corpus retrieval, primary setting.** uses the full generated corpus with all the data points belonging to all the queries to realistically test the RAG system. The corpus contains the gold chunks for the current query, its hard distractors, and all chunks generated for the other query plans in the same split.
- **Query-local retrieval, diagnostic ablation.** Run the same methods on only the chunks generated for the current query: gold chunks, hard distractors, and query-specific background negatives. This tests reranking behavior after the query-local universe has already been isolated.

3 Data splits

- **Validation split:** used after the generation pipeline is fixed. We use it to choose retrieval hyperparameters, especially MMR and facility-location λ values

- **Test split:** final evaluation

4 Prompt Families

Stage	Prompt family	Purpose
Stage 1	Optional catalogue refinement	Usually no LLM is needed. If used, ask for plausible conditions, subgroups, treatments, outcomes, and forbidden combinations from a small seed catalogue. The output should be reviewed manually before use.
Stage 2	Query-plan prompt	Given a catalogue row, produce a hidden plan: primary condition, subgroups, axes, required facets, cluster roles, and difficulty fields. This prompt does not write the final user query.
Stage 3	Structured fact-row prompt	Given one required facet and its cluster role, generate structured synthetic fact rows with <code>must_mention</code> , <code>must_not_mention</code> , metadata fields, and facet IDs.
Stage 4	Chunk-rendering prompt	Convert one or more fact rows into short clinical-style evidence chunks while preserving the assigned facts and avoiding forbidden facts.
Stage 6	Distractor-generation prompt	Given a query plan and distractor type, generate plausible non-gold chunks that are close to the query but fail a specific required facet. Store the explicit <code>distractor_reason</code> .
Stage 7	Query and answer prompts	Generate the natural user query from the hidden plan, then generate the canonical answer from grouped structured facts and fact-to-chunk provenance.
Stage 9	Validation prompts	Use LLM-as-a-judge prompts to check schema compliance, forbidden-fact leakage, distractor leakage, clinical plausibility, and whether the canonical answer uses all required facets. These prompts can reject items.

5 Metrics

Metric	Purpose
Facet Coverage@k	Fraction of required facets represented by at least one retrieved gold chunk. In this reduced pipeline each required facet is implemented as one gold cluster, so this is also cluster coverage. Primary metric.
Weighted Facet Coverage@k	Average fraction of pool-visible gold chunks retrieved per facet.
Gold Precision (GP)	Fraction of retrieved chunks that are gold for any facet.
Gold Recall (GR)	Fraction of pool-visible gold chunks retrieved.
Dominant-Cluster Concentration@k	Fraction of selected chunks from the largest or most query-similar gold cluster. Shows top- k redundancy.
Distractor Rate@k	Fraction of selected chunks that are hard distractors. Shows MMR failure modes.
Jaccard vs. Top- k	Whether reranking materially differs from the baseline.

6 Benchmark Objects

Artifact	Purpose
<code>clinical_facts.parquet</code>	hidden structured source of truth: condition, demographics, comorbidities, interventions, durations, outcomes, complications, and facet IDs.
<code>chunks.parquet</code>	retrieval corpus: <code>chunk_id</code> , <code>fact_ids</code> , <code>facet_ids</code> , <code>cluster_id</code> , <code>distractor_type</code> , and text.
<code>queries.parquet</code>	query text, condition, query type, difficulty, split, and number of facets.
<code>gold_answers.parquet</code>	canonical answer object: <code>answer_text</code> , facet-level answer summaries, answer claims, supporting fact IDs, and supporting chunk IDs.
<code>query_plans.parquet</code>	hidden plan for each query: selected facets, answer outline, split, difficulty, and generation constraints.
<code>qrels.parquet</code>	Gold evidence: query–chunk–facet labels, relevance grade, support type, fact IDs, and optional span offsets.

References

- [1] Zahra Abbasiantaeb, Simon Lupart, and Mohammad Aliannejadi. Generating multi-aspect queries for conversational search. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, pages 8201–8217, 2026. doi: 10.18653/v1/2026.eacl-long.383. URL <https://aclanthology.org/2026.eacl-long.383/>.
- [2] Cristian Cosentino, Annamaria Defilippo, Marco Dossena, Christopher Irwin, Sara Joubbi, and Pietro Liò. Healthbranches: Synthesizing clinically-grounded question answering datasets via decision pathways, 2025.
- [3] Ilias Driouich, Hongliu Cao, and Eoin Thomas. Diverse and private synthetic datasets generation for RAG evaluation: A multi-agent framework, 2025. URL <https://arxiv.org/abs/2508.18929>.
- [4] Lecheng Gong, Weimin Fang, Ting Yang, Dongjie Tao, Chunxiao Guo, Peng Wei, Bo Xie, Jinqun Guan, Zixiao Chen, Fang Shi, Jinjie Gu, and Junwei Liu. Meddialogrubrics: A comprehensive benchmark and evaluation framework for multi-turn medical consultations in large language models, 2026.
- [5] Yue Huang, Siyuan Wu, Chujie Gao, Dongping Chen, Qihui Zhang, Yao Wan, Tianyi Zhou, Chaowei Xiao, Jianfeng Gao, Lichao Sun, and Xiangliang Zhang. DATAGEN: Unified synthetic dataset via large language models. In *International Conference on Learning Representations*, 2025.
- [6] Priyanka Kargupta, Runchu Tian, and Jiawei Han. Beyond true or false: Retrieval-augmented hierarchical analysis of nuanced claims. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 29664–29679, 2025. doi: 10.18653/v1/2025.acl-long.1434. URL <https://aclanthology.org/2025.acl-long.1434/>.
- [7] Tristan Kenneweg, Philip Kenneweg, and Barbara Hammer. RAGVAL: Automatic dataset creation and evaluation for RAG systems. In *2024 2nd International Conference on Foundation and Large Language Models*, pages 470–475, 2024. doi: 10.1109/FLLM63129.2024.10852482.
- [8] Seungone Kim, Juyoung Suk, Xiang Yue, Vijay Viswanathan, Seongyun Lee, Yizhong Wang, Kiril Gashteovski, Carolin Lawrence, Sean Welleck, and Graham Neubig. Evaluating language models as synthetic data generators. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 6385–6403, 2025. URL <https://aclanthology.org/2025.acl-long.320/>.
- [9] Jeongsoo Lee, Daeyong Kwon, Kyohoon Jin, Junnyeong Jeong, Minwoo Sim, and Minwoo Kim. MHTS: Multi-hop tree structure framework for generating difficulty-controllable QA datasets for RAG evaluation, 2025. URL <https://arxiv.org/abs/2504.08756>.
- [10] Junyong Lin, Lu Dai, Ruiqian Han, Yijie Sui, Ruilin Wang, Xingliang Sun, Qinglin Wu, Min Feng, Hao Liu, and Hui Xiong. ScIRGen: Synthesize realistic and large-scale RAG dataset for scientific research. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5619–5630, 2025. doi: 10.1145/3711896.3737432. URL <https://dl.acm.org/doi/10.1145/3711896.3737432>.
- [11] Diego Pastroán, Nicolás Hidalgo, Víctor Reyes, and Erika Rosas. Evolutionary multi-objective prompt learning for synthetic text data generation with black-box large language models. *Applied Sciences*, 16(8):3623, 2026. doi: 10.3390/app16083623. URL <https://doi.org/10.3390/app16083623>.
- [12] Hossein A. Rahmani, Nick Craswell, Emine Yilmaz, Bhaskar Mitra, and Daniel Campos. Synthetic test collections for retrieval evaluation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2647–2651, 2024. doi: 10.1145/3626772.3657942. URL <https://dl.acm.org/doi/10.1145/3626772.3657942>.
- [13] Haiyang Shen, Hang Yan, Zhongshi Xing, Mugeng Liu, Yue Li, Zhiyang Chen, Yuxiang Wang, Jiuzheng Wang, and Yun Ma. DRAGON: Domain-specific robust automatic data generation for RAG optimization. In *Findings of the Association for Computational Linguistics: EACL*, pages 1065–1078, 2026. doi: 10.18653/v1/2026.findings-eacl.56. URL <https://aclanthology.org/2026.findings-eacl.56/>.
- [14] Xiang Yue, Xinliang Frederick Zhang, Ziyu Yao, Simon Lin, and Huan Sun. CliniQG4QA: Generating diverse questions for domain adaptation of clinical question answering. In *2021 IEEE International Conference on Bioinformatics and Biomedicine*, pages 580–587, 2021. doi: 10.1109/BIBM52615.2021.9669300.
- [15] Zafar et al. KI-MAG: A knowledge-infused abstractive question answering system in medical domain, 2024.
- [16] Angelo Ziletti and Leonardo D’Ambrosi. Generating patient cohorts from electronic health records using two-step retrieval-augmented text-to-SQL generation, 2025.